All changes are kept as a sequence of events—in this case, `MessageAdded` events. The current state of the `Conversation` can be rebuilt by replaying the events in the in-memory `Conversation` so it can continue where it left off. This type of database is often called a *journal*, and the technique is known as *event sourcing*. There's more to event sourcing, but this definition will do for now.

What's important to note here is that the journal has become a uniform service. All it needs to do is store all events in sequence and make it possible to retrieve the events in the same sequence as they were written to the journal. (We'll ignore some details for now, like serialization—if you can't wait, look at chapter 9.)

Spreading out the data: Sharding conversations

The next problem is that you're still putting all your eggs in one server. The server restarts, reads all conversations in memory, and continues to operate. The main reason for going stateless in the traditional approach is that it's hard to imagine how you would keep the conversations consistent across many servers. And what would happen if there were too many conversations to fit on one server?

A solution is to divide the conversations over the servers in a predictable way or keep track of where every conversation lives. This is called *sharding* or *partitioning*. Figure 1.11 shows some conversations in shards across two servers.